Recursion/Fixed point
https://en.wikibooks.org/wiki/Haskell/Fix_and_recursion

Recursion
Fixed point
Haskell allows **recursive declarations** due to lazy evaluation
e.g.

```
repeat1 = 1:repeat1
[1,... ]                    -- same
take 2 repeat1              -- OK. [1,1]
```

fix
Ref: https://hackage.haskell.org/package/base/docs/Control-Monad-Fix.html

```
fix :: (a -> a) -> a
fix f = f (fix f)
```

Computes the **fixed point** of a **function f**.
i.e. a point x such that **f(x) = x**.

```
x :: a
x = f x

fix :: (a -> a) -> a          -- equivalent
fix f = let x = f x in x
```

A function with a fixed point **MUST** be **lazy** in its argument.
i.e. the function must not **evaluate** its argument.
e.g.

fix (\x -> 1:x)	OK. x not evaluated
fix (\x -> if x == 0 then ...)	BAD. x evaluated

e.g.

```
repeat12 :: [Int]                       -- GOOD
repeat12 = fix (\v -> (1:2:v))

repeat12' :: [Int]
repeat12' = fix $ \v ->
     if head v == 1 then 2              -- BAD  Cannot evaluate head
v
     else 1):v

print take 4 $ repeat12                 -- [1,2,1,2]
print take 4 $ repeat12'
```

Examples

```
repeat' :: [Int]
repeat' = 1:repeat'          -- recursive
repeat' = fix (1:)             -- same with fix
repeat'                        -- [1,1,...]

fibonacci :: Int -> Int
fibonacci n = if n <=1 then n else fibonacci (n-1) + fibonacci
(n-2)        -- recursive
fibonacci = fix (\f n -> if n <=1 then n else f (n-1) + f (n-2))
         -- with fix
fibonacci 6         -- 8

-- e.g. [1,2,...]
enumFrom' :: Int -> [Int]
enumFrom' start = start:enumFrom' (start + 1)
enumFrom' = fix (\f n -> n:f (n+1))                    -- same
with fix
enumFrom' 0
[0,1,2,...]                  -- same
(enumFrom' 0)!!1          -- OK

-- e.g. factorial
factorial n =
 if n <= 1 then n else n * factorial (n-1)
factorial = fix (\f n ->                                -- same
with fix
 if n <= 1 then n else n * f (n-1))
factorial 5                 -- 120

-- e.g. fibonacci
fibonacci n =
 if n <=1 then n else fibonacci (n-1) + fibonacci (n-2)
fibonacci = fix (\f n ->                                -- same
with fix
 if n <=1 then n else f (n-1) + f (n-2))
fibonacci 0            -- 0
fibonacci 8            -- 8


-- e.g. REPL with IO
runRepl :: Bool -> String -> IO()
runRepl isFirstCall input = do
  let queryNext = do
        putStrLn "Enter next value. Type 'q' to exit"
        nextInput <- getLine
        runRepl False nextInput
      result
        -- case: quit
        | input == "q" || input == "y"
            = do putStrLn "done"
```

```
        -- case: initial input
        | isFirstCall = do
            putStrLn "Start REPL"
            queryNext
        | otherwise = do
            unless (null input) (putStrLn $ "value:" ++ input)
            queryNext
   in result
do runRepl True ""
```

```
Enter next value. Type 'q' to exit
a
value:a
Enter next value. Type 'q' to exit
b
value:b
Enter next value. Type 'q' to exit
q
```

==========

Monad recursion i.e. >>= recursion

Ref: https://hackage.haskell.org/package/base/docs/Control-Monad-Fix.html

Control.MonadFix

```
class (Monad m) => MonadFix m where
   mfix :: (a -> m a) -> m a
```

Monad recursion
Recursive application of >>=
i.e.

x = f x = f (f (f ...)) pure recursion f :: a -> a x :: a

mx = mx >>= f monad recursion f :: a -> m a x :: m a with Monad m
= (...>>= f) >>= f) >>= f) >>= f

i.e. analogue of recursive **function application** (f x) in pure recursion.

Pure recursion x = f x works because **function application** is **lazy**.
Thus **monadic application** mx = mx >>= f must also be lazy for recursion to work.
i.e. depends on the **implementation** of >>=
In fact, in most cases, it is **NOT** lazy.
A separate class **MonadFix** is used to support this special case, if possible.

MonadFix
A special case of monad operation >>= safely handling recursion
i.e. without getting stuck in an **infinite** loop

i.e. implements **mfix** for recursive uses of >>=

```
v = v >>= f            -- unsafe. i.e. usually stuck in infinite
loop
v = mfix f             -- safe
```

MonadFix Laws

```
                   mfix (return . h)  =  return (fix h)
                 Purity
       mfix (\x -> a >>= \y -> f x y)  =  a >>= \y -> mfix (\x -> f
x y)            Left shrinking
                   mfix (liftM h . f)  =  liftM h (mfix (f . h)),
for strict h.     Sliding
      mfix (\x -> mfix (\y -> f x y))  =  mfix (\x -> f x x)
                 Nesting
```

Purity law
For simple cases, we may not need monad recursion
i.e. recursion part is **pure**.
The **purity law** ensures that the result is the same with or without monad recursion.
e.g. infinite list [1...]

```
x = 1:x                                                  -- pure [1...]
x = fix (1:)                                             -- with fix
mx = mfix $ return . (1:)                                -- same with
mfix
mx = do { rec { y <- return (1:y) }; return y }          -- same with
recursive do

-- purity law
mx == return x

-- e.g. use
take 3 x                          -- [1,1,1]
mx >>= return . take 3
```

e.g.

```
instance Monoid a => MonadFix ((,) a) where
    mfix f = let a = f (snd a) in a

instance MonadFix [] where
    mfix f = case fix (f . head) of
                []    -> []
                (x:_) -> x : mfix (drop 1 . f)

instance MonadFix ((->) r) where
    mfix f = \ r -> let a = f a r in a
```

e.g. with list
https://elvishjerricco.github.io/2017/08/22/monadfix-is-time-travel.html

```
mfix $ \x -> do
  a <- [1,2]
  return (snd x, a)       -- recursion in first only

[(1,1),(2,2)]
```

This works due to laziness of snd

```
(snd x, 1) = x    =>    x = (1,1)
(snd x, 2) = x    =>    x = (2,2)
```

[GHC extension] do-rec expression
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/recursive_do.html

RecursiveDo
A GHC language extension
Syntactic sugar on MonadFix to make recursions in monads look similar to pure recursion
e.g.

```
{-# LANGUAGE RecursiveDo #-}

repeat1 :: MonadFix IO
repeat1 = mfix (return . (1:))                          -- with
MonadFix
repeat1 = do
  rec x <- return (1:x)                                 -- with do
rec
  Just x
repeat1 = do { rec { x <- return (1:x) }; Just x }      -- one line
repeat1 = mdo                                           -- with mdo
  x <- return (1:x)
  Just x
repeat1 = mdo { x <- return (1:x) ; Just x }            -- one line


-- analogue in pure recursion
repeat1 = 1:repeat1
repeat1 = fix (1:)
```

Usage

```
-- e.g.list
ml :: MonadFix m => m [Int]
ml = mfix $ return . (1:)          -- [1..]
ml = do                            -- with do rec
```

```
 rec l <- return (1:l)
 return l
ml = do { rec { l <- return 1:l }; return l}         -- one line
-- use
do
 l <- ml
 print (take 3 l)                   -- [1,1,1]
```

```
-- without MonadFix
l :: [Int]
l = 1:l
l = fix (1:)                -- same with fix
ml = return . l
```

```
-- e.g. factorial
mf :: MonadFix m => m (Int -> Int)
mf = mfix (\f ->
 return (\n -> if n <=1 then n else n * f (n-1)))
mf = do
 rec f <- return (\n -> if n <=1 then n else n * f (n-1))     --
with do rec
 return f
-- use
do
 f <- mf
 print (f 5)                -- 120
```

```
-- without MonadFix
f :: Int -> Int
f n = if n <=1 then n else n * f (n-1)
mf = return . f
```

==========

== OLD ==

	Pure recursion	Monad recursion
	`recurse :: a -> a` `result :: a -> a`	`recurse :: MonadFix m => a -> m a` `result :: MonadFix m =>  m a`

expression	-- OK because function application is lazy result = recurse result	-- Theoretically what we want, but doesn't work in practice -- i.e. >>= is not lazy result = result >>= recurse -- NO. -- Need MonadFix instead result = do -- v <- recurse v -- NO. Not supported by default rec v <- recurse v -- OK return v
fix utility	result = fix recurse	result = mfix recurse

```
---- Recursive bindings
--
--
repeat' :: [Int]
repeat' = 1:repeat'
repeat'
[1,1,1...]                -- same
repeat'!!0                -- OK


-- GOTCHA: lazy evaluation failure
-- i.e. never terminates
myEnumFrom0 :: [Int]
myEnumFrom0
  | null myEnumFrom0  = [0]                                -- BAD.
Cannot check empty
  | otherwise = (head myEnumFrom0 + 1:tail myEnumFrom0)
-- solution: use a function instead.
myEnumFrom :: Int -> [Int]
myEnumFrom start = start:myEnumFrom (start + 1)
myEnumFrom0 = myEnumFrom 0
```

```
(x:xs) = myEnumFrom0
      = (x+1):x:xs  in (x:xs) = myEnumFrom
```

***) fix: fixed point**

```
fix :: (a -> a) -> a
fix f = f (fix f)
```

fix
Represents a **fixed point f(x) = x**

```
x :: a
x = f x

fix :: (a -> a) -> a
fix f = let x = f x in x
```

```
---- Data.Function.fix: fixed point function
--
-- i.e. a shorthand for a recursive expression
--
fix :: (a -> a) -> a
fix f = let g = f g in g       -- dfn
x = fix f
x = f x                        -- same
```

Examples

```
myRepeat1 :: [Int]
myRepeat1 = 1:myRepeat1          -- recursive
myRepeat1 = fix (1:)             -- same with fix
myRepeat1                        -- [1,1,...]

fibonacci :: Int -> Int
fibonacci n = if n <=1 then n else fibonacci (n-1) + fibonacci
(n-2)        -- recursive
fibonacci = fix (\f n -> if n <=1 then n else f (n-1) + f (n-2))
         -- with fix
fibonacci 6         -- 8

-- e.g. [1,2,...]
myEnumFrom :: Int -> [Int]
myEnumFrom start = start:myEnumFrom (start + 1)
myEnumFrom = fix (\f n -> n:f (n+1))                    -- same
with fix
myEnumFrom 0
[0,1,2,...]                  -- same
(myEnumFrom 0)!!1          -- OK

-- e.g. factorial
factorial n =
 if n <= 1 then n else n * factorial (n-1)
factorial = fix (\f n ->                                -- same
with fix
 if n <= 1 then n else n * f (n-1))
factorial 5                 -- 120

-- e.g. fibonacci
fibonacci n =
```

```
 if n <=1 then n else fibonacci (n-1) + fibonacci (n-2)
fibonacci = fix (\f n ->                                    -- same
with fix
 if n <=1 then n else f (n-1) + f (n-2))
fibonacci 0            -- 0
fibonacci 8            -- 8


-- e.g. REPL with IO
runRepl :: Bool -> String -> IO()
runRepl isFirstCall input = do
  let queryNext = do
        putStrLn "Enter next value. Type 'q' to exit"
        nextInput <- getLine
        runRepl False nextInput
      result
        -- case: quit
        | input == "q" || input == "y"
            = do putStrLn "done"
        -- case: initial input
        | isFirstCall = do
            putStrLn "Start REPL"
            queryNext
        | otherwise = do
            unless (null input) (putStrLn $ "value:" ++ input)
            queryNext
   in result
do runRepl True ""
```

```
Enter next value. Type 'q' to exit
a
value:a
Enter next value. Type 'q' to exit
b
value:b
Enter next value. Type 'q' to exit
q
```